Undefined Thesis Paper Project

A Thesis
Presented to
The Division of Mathematical and Natural Sciences
Reed College

In Partial Fulfillment
of the Requirements for the Degree
Bachelor of Arts

Riley Wood

May 2026

Approved for the Division
(TBD)

Greg Anderson

Angélica Osorno

Contents

1 Introduction to Formal Verification **1**
1.1 The Setting 1
1.1.1 The Abstraction Function 3
1.1.2 Abstract Domains 4
1.2 The Abstract Lattice 6
1.3 I actually define things 7
1.4 Abstract Transformers 12

2 Abstract Domains **15**

3 The First **17**
3.0.1 References and Labels 17
3.0.2 Footnotes and Endnotes 17
3.1 Bibliographies 17
3.1.1 Tips for Bibliographies 18

4 Tables and Graphics **19**
4.1 Tables 19
4.2 Figures 20
4.2.1 Common Modifications 20

Conclusion **21**
4.1 More info 21

A The First Appendix **23**

B The Second Appendix, for Fun **25**

Bibliography **27**

Chapter 1

Introduction to Formal Verification

1.1 The Setting

I am going for a pretty gentle introduction. Maybe too gentle? I am thinking this will be the part of my thesis my mom reads to support me and I want it to make sense. We'll maybe tackle the Galois connection later

Given that we generally write programs to perform some task, it may be desirable to know that a program does what we want it to. We are interested in demonstrating this via a proof (in the mathematical sense) that a program is "correct". Frequently testing of code is used to determine that it is "correct", but for any program that say, takes an integer as input, there will be a (countably) infinite number of test cases. No amount of testing can verify that there is not some input you have not tried yet which breaks the code. Testing code can only ever provide a proof of incorrectness, that is an example of an input where the code behaves badly.

Acquiring a proof of correctness, or a demonstration that there is no input on which our program behaves incorrectly, requires more ingenuity.

talk about Rice's theorem?

The first challenge is defining what "correctness" refers to. We shall take the notion that a program is correct if for every input, it is impossible to reach an error state.

create a notion of program state (definition here)

formal definition of correctness

With these tools in hand, let us consider the following example

```
x = false
if x :
  kill
```

decide on a simple language and how to display code in the thesis

labels for lines

Here an error will be considered any run of this program in which `kill` is reached.

fix kill not rendering properly

It is quickly evident that this program can never reach the `kill` command, but let us explore how our notion of states applies here.

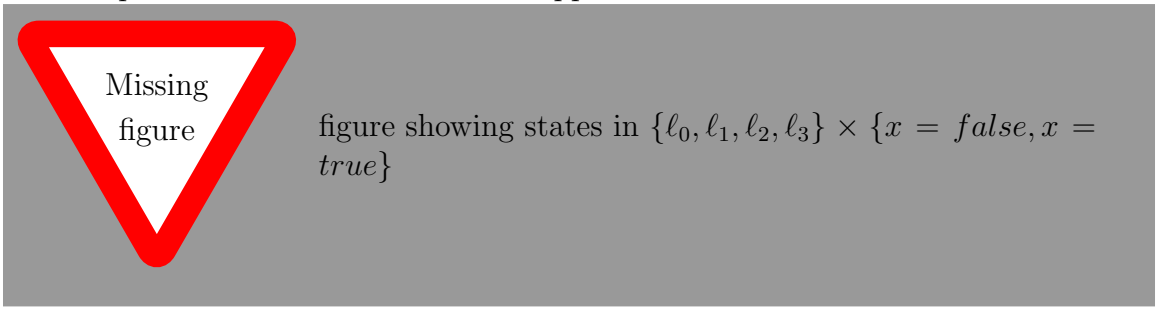


figure showing states in $\{\ell_0, \ell_1, \ell_2, \ell_3\} \times \{x = false, x = true\}$

This state diagram shows all possible program states, and we can construct the arrows between states algorithmically. For a particular state, run the next line of code, and draw an arrow to whatever state you reach. Do this for every possible state, and if there exists a path from the start state to any error state, the program fails verification.
This is the basic idea underpinning Kripke structures, a graph-based approach to checking program correctness.

define Kripke structures

The major limitation of Kripke structures is demonstrated by the following program

```
Assert x >= 0
Assert y >= 0
if x+y < 0:
  kill
```

Despite it being evident that reaching `kill` is not possible, Kripke structures as we have defined them will give us some trouble. The issue comes with the construction of the state diagram for this program, algorithmically constructing a state for every one of the infinite possible starting combination (x, y) will exceed memory limitations. The problem is that the language of Kripke structures is unable to accommodate the possible starting configurations, even though it is clear to our eyes that two positives numbers could not add to a negative number.

I don't actually know if this is a real issue with Kripke structures here. I like this program as an example for sign analysis, but I may be under representing Kripke. Need to get a proper def of Kripke structures (or drop them entirely)

To navigate this, we take an abstract view of each program variable. Rather than think of variables as tied to specific values "x is 1", we can instead consider "x is positive". This is the fundamental idea of abstract interpretation, rather than tracking specific values at each point in the program flow, we track some generalization of the values.

In this example we can apply the idea of sign analysis. We can consider if it is

possible for each variable to be negative, 0, or positive. Say α is our function which takes a variable to its abstraction. At ℓ_0 in the above program, x can be 0 or positive, so we write $\alpha(x) = \{0, +\}$. This allows us to reduce our infinite possible starting configurations down to just one:

$$\ell_0,\ \alpha(x) = \{0, +\},\ \alpha(y) = \{0, +\}.$$

Now we can think about how to perform the check $x+y < 0$. Because we are thinking of x and y in terms of abstractions of their actual values, we need to define $+$ on those abstractions. Fortunately, in this example that is straightforward:

+	-	0	+
-	-	-	{-, 0, +}
0	-	0	+
+	{-, 0, +}	+	+

really this should be a figure

Now we can compute $\alpha(x) + \alpha(y)$ by plugging all the pairs of symbols in x and y into this table and collecting all of the results in a new set:

$$\alpha(x) + \alpha(y) = \{0, +\}.$$

Now we can see that at ℓ_1, $x + y$ cannot take any negative value, allowing us to conclude that `kill` is never reached.

It may feel that I have restated the obvious in describing the possible values that x and y have taken on here, but the importance of this maneuver is that an abstraction α can be performed on a variable by a computer. This enables a computer to automatically perform the reasoning work that we did when we thought "x can't be negative". In general, abstract interpretation seeks to perform this type of work: allowing a computer to more easily make generalizations about the objects in the code we are reasoning about.

1.1.1 The Abstraction Function

As we saw in the previous example, the "abstraction function" α is critical for providing additional reasoning capabilities to the computer. In general, α is thought of as taking program variables and giving us an element in some "Abstract Domain". What we hope is that the elements in our domain will in some way be an overapproximation of the actual values. In our example, this domain consisted of possible combinations of $-, 0$, and $+$, and it is easy to see how we get an overapproximation from this choice. If x could be positive, we have $\alpha(x)$ contain "+", which represents all positive numbers, surely a good approximation for any particular positive number.

need to break this up a little

To make this notion of overapproximation formal, it is first useful to introduce the opposite operation, a concretization fucntion γ. This function takes something

in the abstract domain and tells us what "real values" it could have. In our example above, we could consider

$$\gamma(\{0\}) = \{0\} \quad \text{and} \quad \gamma(\{+\}) = \{1, 2, 3, \ldots\}.$$

With γ in hand we can describe the idea of overapproximation as follows:

$$x \subseteq \gamma(\alpha(x))$$

That is, if we take x and abstract it, then the concrete values which $\alpha(x)$ could take should include the actual value of x.

The other desired effect of α is to take some operation like $+$ and give us a version which works on elements of the abstract domain. If we let c represent some statement in the program, we are interested in an abstract version of that statement $[\![c]\!]$. The idea is that given some program state, we can apply c to get to another program state. Then what we would like is that for a state s, we can abstract it to $\alpha(s)$, and have

yuck this paragraph needs work

$$c(s) \subseteq \gamma([\![c]\!](\alpha(s))).$$

In common language, if we take some program state s and abstract all of the variables in that state to get $\alpha(s)$, then applying the abstract version of c should give us an overapproximation of simply applying c to s.

I don't really like how I am approaching this part. Maybe I should write the formal introduction to these concepts before this?

1.1.2 Abstract Domains

With the work of α clarified, we move our discussion to where alpha takes our program variables to. The abstract domain is where our approximations of variables live, and the example of sign analysis uses subsets of $\{-, 0, +\}$ as our abstract domain. It is clear that this domain gives us an overapproximation that is correct, but we may run into situations where this domain overapproximates too much for us to prove any useful properties. Consider the following program

```
Assert x >=2
if x == 1:
  kill
```

In this program it is clearly impossible for `kill` to be reached, but

$$\alpha(x) = \{+\} \quad \text{so} \quad 1 \in \gamma(\alpha(x)).$$

Thus the check `x == 1` could appear true because our abstract domain thinks that 1 is a possible value of x. The problem here is that our domain is only able to record

that x is positive, so that the information $x >= 2$ is approximated by $x > 0$. This program would then fail to be verified by sign analysis.

In this case, we can resolve the issue by moving to a more expressive domain. Rather than considering only the sign of x, we can let α give us an upper and lower bound on the values of x. In this case,

$$\alpha(x) = [2, \text{inf}]$$

think a little bit about your choice of brackets here

This domain allows us to see that

$$\gamma(\alpha(x)) = \gamma([2, \text{inf}]) = \{2, 3, \ldots\}$$

which does not contain 1. This allows us to verify that `kill` is never reached.

This has demonstrated how our choice of abstract domain can change which programs will pass verification. With a more expressive domain, we get a finer overapproximation, and hence more programs will be verifiable. Coupled with this is an increased computational cost associated with more expressive domains. In the case of intervals, we are now keeping track of two bounds per variable, and computations $[\![c]\!]$ must take both bounds into account.

do I want to make and example of how $[\![+]\!]$ works here?

Thinking about the domain of intervals allows us to verify additional programs, but it is still an overapproximation, and we can construct examples which exploit this. The interval domain is unable to account for relationships between variables, as is demonstrated by the following example program.

```
Assert 0 <= x <= 1
y = x + 1
if y == x:
  kill
```

After y is assigned

once code labels are sorted out, make this more precise

, our abstracted variables are

$$\alpha(x) = [0, 1] \quad \text{and} \quad \alpha(y) = [1, 2].$$

These intervals are correct in that x and y must fall within these ranges, but we lose information about their relationship. Thus when the equality check `y == x` is conducted, we look at the two intervals and see that indeed we could have $x = 1$ and $y = 1$. This causes our program to fail verification.



figure showing x and y's relationship, with a shaded square representing where they could be according to the interval domain, and a line representing where they could actually be

The check $x == y$ will always be false, but our overapproximation is too coarse, leading to this program failing according to our abstract analysis.

This problem is characteristic of abstract interpretation. Because our goal is to overapproximate, there will always be cases where our approximations are too large, and programs will fail verification even though they are correct. Just choosing finer approximations This indicates that we should always choose the finest possible approximation in order to verify as many programs as possible. The challenge is one of efficiency, as we briefly discussed before, the interval domain comes at a slightly higher performance cost than the sign domain. In general, a finer approximation requires us to store more information, leading to worse performance. This leads us to a tradeoff at the heart of abstract interpretation: by picking a domain which more closely approximates the actual values of program variables we are able to correctly verify more programs, however we also must pay more in terms of performance.

Programs frequently want to have some kind of relationship between variables, and the inability to model this is seen as too great a limitation for the interval domain to be particularly useful. In order to capture a finer overapproximation, we can consider a number of other candidate domains, although further exploration of those domains is left to

put furthur discussion of zonotopes and polyhedra (maybe even octagons) somewhere else

1.2 The Abstract Lattice

Despite the faults of the interval domain, it is instructive to explore some further properties of abstract interpretation in the context of intervals. As an example, consider the following program

```
Assert 0 <= x <= 2
if x <= 1 :
  y = 1
else:
  y = 0
if y == 1:
  kill
```

What are the possible values of y when we check `y == 1`?

1.3 I actually define things

In order to treat the abstraction of a program formally, it is first important to create a formal structure for the concrete version of the program.

Definition 1.3.1. *The* concrete lattice L_c *consists of maps from program variables to sets of possible valuations, ordered by set inclusion on each variable.*

As an example, for a program with integer variables, this lattice will consist of maps

$$L_c = \text{Vars} \to 2^{\mathbb{Z}}.$$

this can also be presented as $2^{\text{Vars}\to\mathbb{Z}}$

We order two maps $m \leq n$ if for all variables $v \in ()$, we have that $m(v) \subseteq n(v)$ [1].

prove this is a lattice?

Within this lattice we may be interested in computing certain operations. The standard lattice operations of meet and join will be useful to us, and here they are straightforward to define as intersection and union. We shall soon discuss other lattice operations, as well as how we can use the concrete lattice for verification.

what am I saying in this paragraph?

In order to follow code through its execution, we must have a formal notion of taking a "snapshot" of the values at a particular moment. This is captured by recording a location in the program, that is a label ℓ_i (which for the purpose of this text will be annotations). We also need to capture the value which each variable takes, a task for which our lattice can prove helpful.

Definition 1.3.2. *A* program state $\omega = (\ell, m)$ *consists of a location* ℓ *in the program, and a lattice element* m*.*

We write

$$\omega(x) = b \quad \text{where} \quad b \in 2^{\mathbb{Z}}$$

to say that the map m of state ω assigns the set of values b to variable x. We write $\ell(\omega) = \ell_i$ to say that ω has the location ℓ_i. We write $m(\omega) = m$ to say that ω has the lattice element m.

not sure how I want to handle this notation.

We distinguish one state ω_{init} as the starting configuration of the program. This state must have location ℓ_0, the first label in the code, and will map all variables to the set of possible valuations at the start of execution. It is worth noting that there are a potentially infinite family of initial states. For example if we begin the program with `x` assigned to 0 or 1, then there is an initial state with $\omega(x) = \{0, 1\}$ but also $\omega(x) = \{0, 1, 2\}$ and so on. Similarly, there is a family of of initial states which capture subsets of the possible initial valuations, in our example: $\omega(x) = \{0\}$

and $\omega(x) = \{1\}$. To resolve this we choose ω_{init} to be the least upper bound of all initial configurations.

For some subset of our code c, we might be interested in what states we can get to by running c. We write $\omega[\![c]\!]\nu$ to denote that we can run c starting in state ω and it will terminate in state ν. This only makes sense if the start location of the code c is compatible with ω's associated location. For this triple $\omega[\![c]\!]v$ to be valid, we require $\ell(\omega)$ to equal the start location of c.

Consider

```
x = 3
x += 2
x = x-1
```

Our minimal possible program state at ℓ_1 is

$$\omega = (\ell_1, x \mapsto \{3\})$$

This tells us that 3 is the value of x at this location in code. Now consider how the semantics of the line `x += 2` will affect this state. We can see that

$$\omega[\![\texttt{x += 2}]\!] = (\ell_2, x \mapsto \{5\}).$$

This tells us that the semantics of `x += 2` relate a state with x assigned the value 3, to a state where x is assigned 5.

In this example, it seems as if $[\![c]\!]$ acts as a function on program states, but this notion can break down, as in the following example:

```
x = 3
while (x == 3): pass
x = x-1
```

Now consider how line 2 affects our program state. As above, we begin with $\omega = (\ell_1, x \mapsto \{3\})$, but now there is no valid state ν to complete

$$\omega[\![\texttt{while(x == 3):pass}]\!]\nu.$$

Because there is no valid computation of line 2 which terminates, there can be no terminal state ν to complete the triple. This makes it clear that we cannot view $[\![c]\!]$ as a function on program states, as there can be some states for which $\omega[\![c]\!]$ is undefined

We could instead imagine $[\![c]\!]$ as a partial function i.e. one which does not evaluate on some inputs, but this too has problems:

```
x = 3
x ~ {0, 1}
```

Here we see x sampled nondeterministically from $\{0, 1\}$. If we think about how the semantics of the second line treats the program state $\omega = (\ell_1, x \mapsto \{3\})$, we see that there are several possible mappings by $[\![\texttt{x 0, 1}]\!]$.

Why does this code example not work?

$$\omega[\![\texttt{x := 0, 1}]\!](\ell_3, x \mapsto \{0\}) \quad \text{and} \quad \omega[\![\texttt{x := 0, 1}]\!](\ell_3, x \mapsto \{1\})$$

are both valid (as is $x \mapsto \{0, 1\}$). Here we see how semantics can send one state ω to multiple other states. To capture this behavior alongside the possibility of ω being sent to no other state, we define semantics as a relation on program states [1].

Definition 1.3.3. *The* concrete semantics *of a command c are a relation on program states where $\omega \leq \mu$ if there is a computation $\omega[\![c]\!]\nu$.*

is command what I want to phrase this as?

In particular, if ω and ν have locations ℓ_i and ℓ_{i+1} respectively, the relation $[\![c]\!]$ tells us how the line of code between those locations affects the programs variables Notice that the relation given by some semantics $[\![c]\!]$ also induce a relation on lattice elements, where $l(\omega) \leq l(\mu)$ if and only if $\omega \leq \mu$.

do I do anything with this??

With this groundwork in place, we can define program correctness. Earlier we thought of correctness as "not doing anything bad" but we can now characterize this notion in terms of a set of error states. We can think of error states as problems that will occur when running the program i.e. dividing by 0, or problems with the output of the program i.e. computing something incorrectly.

talk about infinite loops and lassos?

These are both captured by the following definition:

Definition 1.3.4. *Given a program and some set of error states Ω, we say that* the program is correct *if for all subsets c of the program, there is no state $\nu \in \Omega$ such that $\omega_{init}[\![c]\!]\nu$ is valid.*

"valid" isn't really defined. also subsets of a program doesn't make sense. Think about how to split up the semantics.

Now we can see why verification is so challenging, the number of program states is tremendous, and the possible combinations of semantics which must be checked grow exponentially. For this reason, we wish to reduce the number of states we must check, and the burden of program semantics

I think I need to sit with the idea of semantics of just part of a program and figure out what the heck I mean. Probably related to Greg's comment about the meaning of a program being captured by the least fixed point of its semantics

To do so, we abstract the concrete lattice L_c to some abstract lattice L_a where we hope proving things will be somewhat easier. The idea is to capture an overapproximation of each variable's possible values, and follow those overapproximations through the program. This means that instead of testing the program on every valuation x can take, we instead allow the computer to follow the range of values for x

through the program. To affect this abstraction, we define a function α which will contain information for mapping L_c to L_a. This abstraction must be paired with some concretization function that maps the other direction.

this should probably be replaced with better narrative

Definition 1.3.5. *An* abstraction function α *and a* concretization function γ *are adjoint maps between the concrete lattice* L_c *and the abstract lattice* L_a*. Where* $\alpha : L_c \to L_z$ *and* $\gamma : L_a \to L_c$*.*

As an example, we could consider L_c to be supersets of $\mathbb{Z}$, and L_a to be intervals. In this case we might have

$$\alpha(\{0\}) = [0], \quad \alpha(\{1,2\}) = [1,2], \quad \text{and} \quad \alpha(\{1,3,5,6\}) = [1,6].$$

We see that alpha takes elements of our concrete lattice, and gives us intervals of integers instead. This means that we need not track how the semantics affect each value in a set, but rather just the endpoints of the interval, significantly easing the burden of checking how our code behaves.

Similarly we can have γ map the other way

$$\gamma([0]) = \{0\}, \quad \text{and} \quad \gamma([0,2]) = \{0,1,2\}.$$

It is important that α have some notion of being a "good" overapproximation, that is

$$\alpha(\{1,2,3\}) = [1,2]$$

should be viewed as incorrect as we fail to capture all of the possible values in the concrete lattice. This is captured by the adjunction which ensures order preservation when mapping between our lattices.

I would like to handle the category theory a little more carefully here

$$\text{for all} \quad x \in L_c, \quad \text{and} \quad y \in L_a,\ x \sqsubseteq \gamma(y) \iff \alpha(x) \sqsubseteq y.$$

This is better captured by the diagram

$$\begin{array}{ccc} \gamma(y) & \xleftarrow{\quad\gamma\quad} & y \\ \sqsubseteq & & \sqsubseteq \\ x & \xrightarrow{\quad\alpha\quad} & \alpha(x) \end{array}$$

This tells us that the ordering of the concrete lattice is always preserved when we abstract and concretize our values.

Given the abstract lattice, it is natural to adjust our notion of program state to include abstraction.

Definition 1.3.6. *An* abstract state $\omega^\# = (\ell_i, m^\#)$ *consists of a location* ℓ_i *in the program, and an element* $m^\# \in L_a$.

Given a program state $\omega = (\ell_i, m)$, it will often be helpful to consider it's "abstracted version", for which we shall write

$$\omega^\# = (\ell_i, \alpha(m)).$$

Also similar to our previous definitons, it will be important to capture how the semantics of the program behave in the abstract lattice. Consider the following program:

```
Assert 1<x<6
x = x-1
```

In this example, we can see $\omega_{\text{init}} = (\ell_1, x \mapsto \{2,3,4,5\}$, so that $\omega^\#_{\text{init}} = (\ell_1, x \mapsto [2,5])$. Then we would like to compute what abstract state $\nu^\#$ completes $\omega^\#_{\text{init}}[\![\texttt{x = x-1}]\!]^\#\nu^\#$.

Definition 1.3.7. *The abstract semantics* $[\![c]\!]^\#$ *of a program part* c *give a relation on abstract program states, where* $\omega^\# \leq \nu^\#$ *iff* $\omega^\#[\![c]\!]^\#\nu^\#$.

This definition on its own is perhaps not particularly helpful, but we also combine it with an idea of soundness, that is that our abstract semantics should always preserve our overapproximation. Formally, for any element m of the concrete lattice with an abstraction

$$\alpha(m) \sqsubseteq a \in L_a,$$

when we apply some semantics to m and arrive at a new concrete element n

$$m[\![c]\!]n,$$

we want the abstract semantics $[\![c]\!]^\#$ to map a to abstract states b that are overapproximations of n:

$$a[\![c]\!]^\# b \quad \text{where} \quad n \sqsubseteq \gamma(b).$$

Once again this is easier to capture in a diagram

$$\begin{array}{ccc} \gamma(a) & \xrightarrow{\quad [\![c]\!]^\# \quad} & \gamma(b) \\ \sqsubseteq \big| & & \big| \sqsubseteq \\ m & \xrightarrow{\quad [\![c]\!] \quad} & n \end{array}$$

This says that if we begin with a valid overapproximation and apply the abstract syntax of any part of our program, we should end up with a valid overapproximation of how that syntax behaves in the concrete world.

Now we have the tools to understand abstract interpretation. The idea is to give adjoint functions α and γ which take us to and from the abstract lattice. We then define abstract semantics $[\![c]\!]^\#$ for each command c in our language. Then we can abstract the initial state, and begin stepping through the code, checking after each line to see if our abstract state overapproximates any concrete error states. This allows us to test every possible input combination in a single run of the program.

1.4 Abstract Transformers

Remaining is the challenge of picking a suitable abstract lattice, and defining the abstract semantics for certain commands $[\![c]\!]^\#$. We shall tackle the second issue, and defer abstract lattices to a later chapter.

We have seen examples in the interval domain where the abstract semantics of adding 1 to a variable increases both endpoints of our abstract element. In general, intervals are quite good for capturing mathematical operations of this sort, as we have a relatively easy time programming how operations like multiplication, addition, and subtraction will affect a range of possible values. Consider, however, the following program:

```
Assert 0 < x < 5
if (x == 1):
  x = 10
else:
  x = -x
```

This code begins with $\alpha(x) = [1, 4]$, an abstraction which intervals capture quite tightly, but then we encounter a control flow concept. In order to find the possible values of x after this `if` statement, we must follow all paths that the program could take. This requires us to split into two interpretations, one for the branch `x=10` and one for `x = -x`.

To do this formally, if may be useful to think of defining a transformer for a general if statement of the form `if c: x else: y`, where `x` and `y` are parts of the program. We can then think about splitting up our program into two branches. The `if` branch should start in the initial state ω subject to the condition that c is true, and then perform interpretation of `x`. The starting state subject to the condition that c is true can be denoted $\omega \sqcap c$ where c captures the least upper bound of states that make c true. We can notate this as

$$(\omega \sqcap c)[\![\texttt{x}]\!]\nu$$

To say that the first branch lands us in state ν. Similarly for the else branch we get

$$(\omega \sqcap !c)[\![\texttt{x}]\!]\nu$$

$$\omega[\![\texttt{if c: x else: y}]\!] = (\omega \sqcap c)[\![\texttt{x}]\!] \sqcup (\omega \sqcap !c)[\![\texttt{y}]\!]$$

Following the first branch takes us to ℓ_4, where we know that x can only be 1. The conditional can be satisfied, as $1 \in [1, 4]$, so we can follow this branch, but while doing so we can narrow our abstraction so that x is in the interval [1]. Now we can apply the semantics of line 3 to get x in [10].

Similarly for the `else` path, the conditional narrows our interval to $[2, 4]$, and then the semantics of line 5 tell us that $\alpha(x) = [-4, -2]$.

But now we have a problem, after this control flow ends we reach ℓ_7 where the two paths must recombine, but their intervals are not compatible. We know that x lives in one of $[-4, -2]$ or [10] but not which one. To continue with just 1 case to

check, we must find a new overapproximation of these two intervals. This requires us to compute the join in L_a, in this case $[-4, -2] \sqcup [10] = [-4, 10]$. Now we once again have a single overapproximation, but it has become a far less accurate approximation of the actual values of x. Control flow operations almost always require us to compute the join of our abstract elements, frequently to the detriment of our approximation's quality.

Another challenge we face when performing abstract interpretation is loops. Consider the following example

```
Assert x < 0
while (x<1):
  x = x+1
```

How does our Abstract Interpreter handle this?

Chapter 2

Abstract Domains

Our motivating example has also introduced a classic domain for abstract interpretation.

Definition 2.0.1. *The* interval domain *consists of intervals over* $\mathbb{R} \cup \{-\infty, \infty\}$.

For this domain, our abstraction function α sends a variable x to the range of possible values it could take, and semantics c to some notion of how the semantics could affect the endpoints of the intervals.

This needs some work. Maybe α should be included in the abstract domain def?

Definition 2.0.2. *The* Zonotope domain *consists of noise symbols* n_i *with each associated to an interval over* $\mathbb{R} \cup \{-\infty, \infty\}$. *These intervals are restricted to* $[-1, 1]$, $[-1, \infty)$, *or* $(-\infty, 1]$.

The expressive power of this domain lies in how α maps program variables to the domain. We let

$$\alpha(x) = a_0 + \sum_{i=1}^{d} a_i n_i$$

where d is the number of noise symbols defined (also the dimension of the space in which our noise symbols live), and each $a_i \in \mathbb{R}$. This is an affine transformation on the noise symbols, which allows us to capture linear relationships between variables.

work an example here?

Being able to easily capture linear relationships between variables allows analysis with the Zonotope domain to verify more programs than the interval domain can handle. Not all linear relationships can be handled by this model though.

work an example here

The way that we are able to capture relationships between variables is through exploiting the noise symbols. When sampling a variable, we fix a possible value or range of values for each noise symbol, this gives us a restriction on the possible values of other variables to those values that can occur under the restricted range of noise symbols.

This is a bad explaination, and probably better handled through example, but it needs to be said somewhere

Definition 2.0.3. *The* polyhedra domain

How do we want to approach this?

Definition 2.0.4. *A* Probabilistic Polyhedron *consists of*

1. *Noise symbols* $(\varepsilon_1, \ldots, \varepsilon_n) = \vec{\varepsilon} \in (\mathbb{R} \cup \{-\infty, \infty\})^n$
2. *A map* M *from program variables to affine forms over* $\vec{\varepsilon}$
3. *A map* Δ *from noise symbols to probability distributions over* $\mathbb{R} \cup \{-\infty, \infty\}$
4. *A set of linear constraints described by a* $m \times n$ *matrix* A *and a* $m \times 1$ *vector* z *such that* $A\vec{\varepsilon} \leq z$.

Give the concretization

Once again we also allow α to map variables via an affine transformation on the noise symbols. The key difference from the domain of probabilistic zonotopes is that we allow arbitrary constraints between noise symbols described by A and z. This means that rather than our affine map taking a "box" of possible noise symbol values and transforming them, it now takes an arbitrary polyhedron of noise symbols.

Chapter 3

The First

3.0.1 References and Labels

Sometimes you'd like to refer to a table or figure, e.g. you can see in Figure **??** that you can rotate figures . Start by labeling your figure or table with the label command (`\label{labelvariable}`) below the caption (see the chapter on graphics and tables for examples). Then when you would like to refer to the table or figure, use the ref command (`\ref{labelvariable}`). Make sure your label variables are unique; you can't have two elements named "default." Also, since the reference command only puts the figure or table number, you will have to put "Table" or "Figure" as appropriate, as seen in the following examples:

As I showed in Table **??** many factors can be assumed to follow from inheritance. Also see the Figure **??** for an illustration.

3.0.2 Footnotes and Endnotes

You might want to footnote something.[1] Be sure to leave no spaces between the word immediately preceding the footnote command and the command itself. The footnote will be in a smaller font and placed appropriately. Endnotes work in much the same way. More information can be found about both on the CUS site.

3.1 Bibliographies

Of course you will need to cite things, and you will probably accumulate an armful of sources. This is why BibLaTeX was created. For more information about BibLaTeX and bibliographies, see our CUS site (`web.reed.edu/cis/help/latex/index.html`)[2]. There are three pages on this topic: *biblatex* (which talks about using BibLaTeX, at `/latex/biblatex.html`), TODO STYLE DOCUMENTATION and *bibman* (which covers how to make and maintain a bibliography by hand, without BibLaTeX, at at `/latex/bibman.html`). The last page will not be useful unless you have only a

[1] footnote text

[2] **reedweb:2007**

few sources. There used to be APA stuff here, but we don't need it since I've fixed this with my apa-good natbib style file.

3.1.1 Tips for Bibliographies

1. Like with thesis formatting, the sooner you start compiling your bibliography for something as large as thesis, the better. Typing in source after source is mind-numbing enough; do you really want to do it for hours on end in late April? Think of it as procrastination.

2. The cite key (a citation's label) needs to be unique from the other entries.

3. When you have more than one author or editor, you need to separate each author's name by the word "and" e.g.
   `Author = {Noble, Sam and Youngberg, Jessica},`.

4. Bibliographies made using BibTeX (whether manually or using a manager) accept LaTeX markup, so you can italicize and add symbols as necessary.

5. To force capitalization in an article title or where all lowercase is generally used, bracket the capital letter in curly braces.

6. You can add a Reed Thesis citation[3] option. The best way to do this is to use the phdthesis type of citation, and use the optional "type" field to enter "Reed thesis" or "Undergraduate thesis". Here's a test of Chicago, showing the second cite in a row[4] being different. Also the second time not in a row[5] should be different. Of course in other styles they'll all look the same.

[3] **noble:2002**
[4] **noble:2002**
[5] **reedweb:2007**

Chapter 4

Tables and Graphics

4.1 Tables

The following section contains examples of tables, most of which have been commented out for brevity. (They will show up in the .tex document in red, but not at all in the .pdf). For more help in constructing a table (or anything else in this document), please see the LaTeX pages on the CUS site.

4.2 Figures

4.2.1 Common Modifications

The following figure features the more popular changes thesis students want to their figures. This information is also on the web at `web.reed.edu/cis/help/latex/graphics.html`.

Figure 4.1: Subdivision of arc segments. You can see that $p_3 = p'_6$.

Conclusion

Here's a conclusion, demonstrating the use of all that manual incrementing and table of contents adding that has to happen if you use the starred form of the chapter command. The deal is, the chapter command in LaTeX does a lot of things: it increments the chapter counter, it resets the section counter to zero, it puts the name of the chapter into the table of contents and the running headers, and probably some other stuff.

So, if you remove all that stuff because you don't like it to say "Chapter 4: Conclusion", then you have to manually add all the things LaTeX would normally do for you. Maybe someday we'll write a new chapter macro that doesn't add "Chapter X" to the beginning of every chapter title.

4.1 More info

And here's some other random info: the first paragraph after a chapter title or section head *shouldn't be* indented, because indents are to tell the reader that you're starting a new paragraph. Since that's obvious after a chapter or section title, proper typesetting doesn't add an indent there.

Appendix A

The First Appendix

Appendix B

The Second Appendix, for Fun

Bibliography

[1] A. Sampson, *Lecture notes on abstract interpretation*, 2017.